

识别手写数字

2025年12月4日 22:10

<https://gemini.google.com/share/e310b097f4d7>

```
#构建网络框架
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(28 * 28, out_features=64),#-----图片是28*28像素，且是黑白图片，只有1层
            nn.Linear(in_features=64, out_features=64),
            nn.Linear(in_features=64, out_features=64),
            nn.Linear(in_features=64, out_features=10),#----输出10个类别
        )

    def forward(self,x):

class Net(torch.nn.Module):

    def __init__(self):
        super().__init__()
        self.fc1 = torch.nn.Linear(28*28, 64)
        self.fc2 = torch.nn.Linear(64, 64)
        self.fc3 = torch.nn.Linear(64, 64)
        self.fc4 = torch.nn.Linear(64, 10)

    def forward(self, x):
        x = torch.nn.functional.relu(self.fc1(x))
        x = torch.nn.functional.relu(self.fc2(x))
        x = torch.nn.functional.relu(self.fc3(x))
        x = torch.nn.functional.log_softmax(self.fc4(x), dim=1)
        return x
```

用sequential 也不一定就是好，比如就没法像后面那样，每一层再套一个非线性

```
def __init__(self):
    super().__init__()
    self.model = nn.Sequential(
        relu(nn.Linear(28 * 28, out_features=64)), #-
        nn.Linea 应为类型 'Module', 但实际为 'Tensor'
        nn.Linea 应为类型 'Tensor', 但实际为 'Linear'
        nn.Linea © torch.nn.modules.linear.Linear
    )
    def __init__(self,
```

为什么不能这样写

分析

这个问题问得好，小张！你正在尝试一种 创新但错误的 写法，它指出了你对 PyTorch 模块 (Modules) 和 函数 (Functions) 的一个关键误解。

你提供的代码片段是想在 `nn.Sequential` 里直接使用 `relu(nn.Linear(...))`，这是 不能这样写 的。

为什么不能这样写？

这个错误的核心在于：`nn.Sequential` 只能接收 PyTorch 的模块 (Module)，而不能接收一个模块的输出结果或一个简单的函数调用。

让我们用你的代码和截图中的提示来具体分析一下：

- `nn.Sequential` 的要求
 - `nn.Sequential` 是一个“有序容器”：它像一条工厂的流水线，你放进去的每一个“工人”都必须是一个完整的、可独立运行的 PyTorch 模块 (`nn.Module`)。
 - 什么是模块？像 `nn.Linear`、`nn.ReLU`、`nn.Conv2d` 这些大写的类名，它们都是模块。它们内部封装了参数 (如权重 W 和偏置 b) 和向前传播的方法。
 - 什么是函数？像 `F.relu` (即 `torch.nn.functional.relu`) 或你在代码中直接调用的 `relu(...)` 这种，它们只是 无状态的数学操作

```
def forward(self,x):
    x = relu(self.f1(x))

'Linear' 对象不可调用
移除调用 Alt+Shift+Enter
practice.num_detach_f1: tuple[Linear] =
```

根本原因 1: 属性定义错误 (最关键)

- 注意你在定义 `f1` 时的逗号: `nn.Linear(28 * 28, 64),`
- 这个 尾随的逗号 使得 `self.f1` 被 Python 视为一个 单元元素组 (tuple)，而不是一个单独的 `nn.Linear` 模块。
- 一个元组对象是 不可调用 的，所以当你尝试 `self.f1(x)` 时，Python 报错 `'Linear' 对象不可调用` (它实际上是在抱怨你试图调用这个元组)。

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.f1 = nn.Linear(28 * 28, out_features=64),#-----图片是28*28
        self.f2 = nn.Linear(in_features=64, out_features=64),#
        self.f3 = nn.Linear(in_features=64, out_features=64),#
        self.f4 = nn.Linear(in_features=64, out_features=10),#----输出10个
```

1. `log_softmax` 和 损失函数的 使命: 指导模型训练

你的代码里是这样做的:

```
Python

x = torch.nn.functional.log_softmax(self.fc4(x), dim=1) # 得到对数概率输出

# 损失计算 (在 main 函数的训练循环中)
loss = torch.nn.functional.nll_loss(output, y) # NLL Loss 依赖 log_softmax 输出
loss.backward()
optimizer.step()
```

- `log_softmax`: 它将网络最后一层的输出 (logits) 转化为 对数概率。它的输出形状是一个 $(Batch_Size, 10)$ 的矩阵, 每一个值代表模型对某个类别的信心度。
- `nll_loss` (负对数似然损失): 这个损失函数会使用 `log_softmax` 的输出, 并结合真实标签 y 来计算一个 标量值 (Loss)。
- 关键点: Loss 的主要目标是提供一个 平滑、可导 的数值。这个数值告诉我们模型的预测 偏离 真实结果的程度, 以及 如何调整 内部的权重 (通过梯度)。Loss 越小, 说明预测结果越接近真实标签。它是优化器进行梯度下降的唯一依据!

分类模型

最后会输出10组数据, 可以当作是十组标签对应的输出值
本身就只是数字, 和模型的输出正确性也没有什么关系
但是我通过训练, 使得他们产生了一定的对应关系,
使数字1输入时, 对应标签1 的数据是最高的.....

再通过归一化, 使得标签对应的十组数字即为模型分辨的
正确率。
我们这里直接把这个数据, 结合loss函数, 作为模型训练的
依据

1. 网络输出: `log_softmax(logits)` → 得到 对数概率。
2. 损失函数: `nn.NLLLoss()` (负对数似然损失)。

在这个 CNN 模型中, 你大概率会使用 PyTorch 中的另一个损失函数:

训练设置	网络输出 (forward 结果)	损失函数
CNN 模型 (你的源代码)	Logits (10个原始得分)	<code>nn.CrossEntropyLoss()</code>
全连接模型 (你的源代码)	Logits 经过 LogSoftmax	<code>nn.NLLLoss()</code>

导出到 Google 表格

真正的巧妙之处在于这里:

PyTorch 的 `nn.CrossEntropyLoss` 内部 已经集成了 `Softmax` 和 `NLLLoss` 这两个步骤!

$CrossEntropyLoss(Logits, Labels) = NLLLoss(LogSoftmax(Logits), Labels)$

为什么这样做更聪明? (数值稳定性)

直接在 Logits 上应用 `nn.CrossEntropyLoss`, 而不是在 forward 中显式地计算 `log_softmax`, 可以避免在计算 Softmax 过程中可能发生的 数值溢出或下溢 问题。

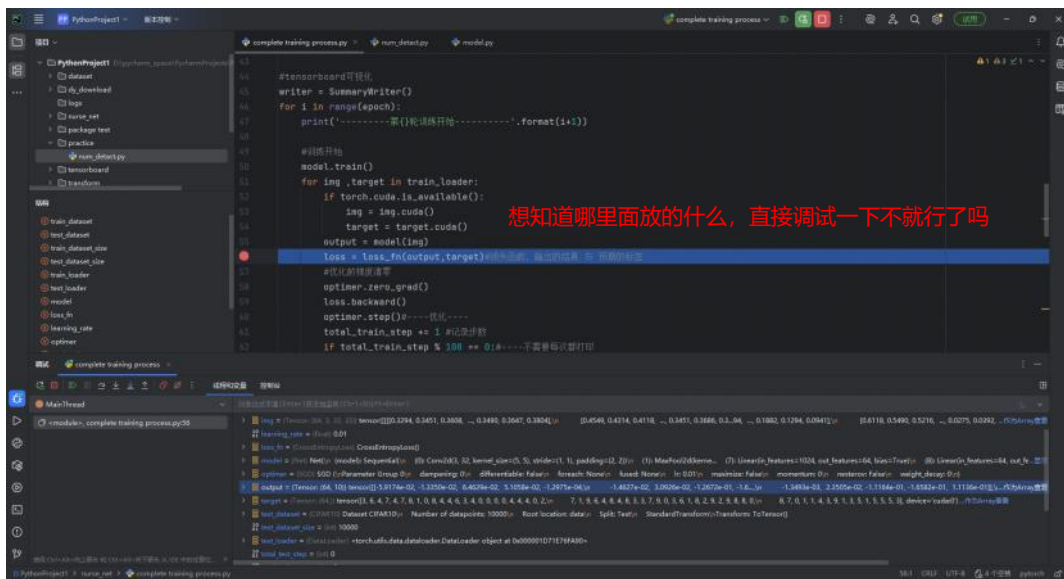
- 官方推荐: PyTorch 官方强烈建议, 当使用 `nn.CrossEntropyLoss` 时, 模型的 forward 函数应该只输出 原始 Logits, 而不进行 `Softmax` 或 `LogSoftmax` 激活。

虽然这个模型里面没有用归一化得到概率

你在纠结, 那我都不知道output里面的数多大, 我直接和target的0, 1, 2。。。比, 有意义吗

但我损失函数用的是交叉熵, 不需要你考虑那么多

想知道哪里放的什么, 直接调试一下不就行了吗



#测试函数，用于测试训练的效果

```
def evaluate(test_data,net):
```

```
    total_loss = 0
```

```
    total_accuracy = 0
```

```
    with torch.no_grad():
```

```
        for(img,target)in test_data:
```

```
            #img是28*28的像素组成的，但网络模型只允许展平了的一维输入
```

```
            #使用x.view()改变形状
```

```
            img = img.view(-1,28*28)#-1的意思是，让它自己根据图片原始像素大小以及要求的输出参数，自动确定维数
```

```
            output = net(img)
```

```
            loss = nn.NLLLoss(output,target)
```

```
            total_loss += loss.item()#由于total_loss是不断累加的，使用要在每一轮开始前清零，不然就失去比较意义了
```

```
            #再来算正确率
```

```
            accuracy = (output.argmax(1)==target).sum#先把正确的个数统计出来
```

```
            total_accuracy += accuracy.item()
```

```
            accuracy_rate = total_accuracy / len(test_data)#算的是每一轮的测试数据集的正确率，下一轮开始前也要把total_accuracy清零
```

Loss函数一般只在训练过程中使用

#主函数

```
def main():
```

```
    train_data = get_data_loader(True)
```

```
    test_data = get_data_loader(False)
```

Test_data的长度是批次数

```
print(f'初始正确率是: {}'.format(evaluate(test_data,net)))
```

你这是什么四不像

应为表达式

Python

```
print(f"初始正确率是: {evaluate(test_data, net)}")
```

或者:

Python

```
print("初始正确率是: {}".format(evaluate(test_data, net)))
```

```
e([transforms.ToTensor()])),
```

这个经常错

Torchvision.transforms.ToTensor()